

TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop (HITL)

Prepared by: Kirthika R S

Internship Project — 2025

1. Introduction

1.1 What is RAG (Retrieval-Augmented Generation)?

Retrieval-Augmented Generation (RAG) is an AI architecture that combines two powerful techniques: information retrieval and text generation. Instead of relying solely on a language model's internal training knowledge, RAG first retrieves relevant information from an external knowledge source (such as a PDF document or database), and then uses that retrieved information as context to generate a more accurate and grounded answer.

In a standard LLM, the model can only answer based on what it learned during training. This leads to outdated answers, hallucinations, and lack of domain-specific knowledge. RAG solves this by dynamically fetching the right information at the time of the query, making responses more reliable and factually grounded.

1.2 Why is RAG Needed?

- LLMs have a knowledge cutoff date and cannot access real-time or domain-specific data
- Without RAG, LLMs often hallucinate facts that sound plausible but are incorrect
- RAG grounds the LLM's response in actual retrieved documents, improving accuracy
- It allows organizations to use their own private documents as a knowledge base
- RAG reduces the cost of fine-tuning by using retrieval instead of retraining the model

1.3 Use Case Overview

This project implements a RAG-based Customer Support Assistant for a fictional company called TechCare, which makes a SmartHome App. The assistant reads a FAQ PDF document, stores it in a vector database, and answers customer queries about password reset, device setup, refund policies, subscriptions, and more. When the bot cannot find a relevant answer, it escalates the query to a human agent using the HITL (Human-in-the-Loop) mechanism.

2. System Architecture Explanation

2.1 Detailed Explanation of HLD

The system is built as a pipeline with two major phases: the Ingestion Phase and the Query Phase.

Ingestion Phase (run once):

- The PDF knowledge base is loaded using PyPDFLoader
- The text is split into overlapping chunks using RecursiveCharacterTextSplitter
- Each chunk is converted to a vector embedding using HuggingFace all-MiniLM-L6-v2
- All embeddings are stored in a FAISS vector index saved to disk

Query Phase (run every time user asks a question):

- User enters a question in the terminal
- The question is embedded using the same HuggingFace model
- FAISS performs similarity search and returns top-3 relevant chunks
- LangGraph routes the query through process_node to answer_node or escalate_node
- The Groq LLM generates a response using the retrieved context
- The final answer is displayed to the user

2.2 Component Interactions

From Component	To Component	Data Passed
main.py	LangGraph graph	Initial State with user query
process_node	retriever.py	User query string
retriever.py	FAISS	Embedded query vector
FAISS	retriever.py	Top-3 matching document chunks
process_node	route()	State with escalate flag
route()	answer_node or escalate_node	Routing decision
answer_node	Groq LLM API	Prompt with context and query
Groq LLM API	answer_node	Generated answer text

escalate_node	hitl.py	User query for human agent
hitl.py	main.py	Human typed response

3. Design Decisions

3.1 Chunk Size Choice

Chunk size was set to 500 characters with an overlap of 50 characters. This was chosen because:

- 500 characters is large enough to capture a full question-answer pair from the FAQ
- It is small enough to ensure precise retrieval without including irrelevant content
- The 50-character overlap ensures that answers split across chunk boundaries are not lost
- Smaller chunks (e.g., 100 chars) would lose context; larger chunks (e.g., 2000 chars) would retrieve too much noise

3.2 Embedding Strategy

The all-MiniLM-L6-v2 model from HuggingFace was chosen for the following reasons:

- It is a lightweight model with only 22 million parameters, making it fast to run locally
- It produces 384-dimensional embeddings that capture semantic meaning effectively
- It is completely free and requires no API key or internet connection at inference time
- It performs well on sentence similarity tasks, which is exactly what retrieval requires

3.3 Retrieval Approach

FAISS was used instead of ChromaDB for the following reasons:

- FAISS does not require C++ build tools, making it easier to install on Windows
- It supports fast approximate nearest neighbor search even for large datasets

- The top-3 retrieval (k=3) balances between getting enough context and avoiding noise
- FAISS saves the index to disk, allowing the system to avoid re-ingestion on every run
-

3.4 Prompt Design Logic

The prompt sent to the Groq LLM follows a structured format:

```
You are a customer support assistant.  
Use the context below to answer the question.  
If you are not sure, say you don't know.
```

```
Context: {retrieved chunks}  
Question: {user query}  
Answer:
```

This prompt design ensures the LLM stays grounded in the retrieved context and does not hallucinate. The instruction to say it does not know prevents false confidence in uncertain situations.

4. Workflow Explanation

4.1 LangGraph Usage

LangGraph is a library built on top of LangChain that allows developers to define AI workflows as directed graphs. Each node in the graph is a Python function that reads from and writes to a shared State object. Edges define the flow between nodes, and conditional edges allow dynamic routing based on the state values.

In this project, LangGraph is used to orchestrate the three-step workflow: process the query, generate an answer, and optionally escalate to a human. This graph-based approach makes the workflow easy to extend, debug, and maintain.

4.2 Node Responsibilities

Node	Function	Input	Output
process	process_node()	query from state	context + escalate flag in state
answer	answer_node()	query + context from state	answer in state
escalate	escalate_node()	query from state	human answer in state

4.3 State Transitions

- State starts with: query set, context empty, answer empty, escalate False
- After process_node: context is filled with retrieved chunks, escalate may be True
- If escalate is False: state moves to answer_node, answer is filled by LLM
- If escalate is True: state moves to escalate_node, answer is filled by human
- Final state contains the complete answer ready to display to user

5. Conditional Logic

5.1 Intent Detection

The system does not perform explicit intent classification. Instead, it uses retrieval confidence as a proxy for intent matching. If the user's query semantically matches content in the knowledge base, FAISS will return relevant chunks with high similarity scores. If the query is outside the scope of the knowledge base (for example, asking about something not in the FAQ), FAISS will return low-relevance or empty results.

This approach is simpler than building a separate intent classifier and works effectively for FAQ-based customer support systems where the knowledge base covers all expected intents.

5.2 Routing Decisions

The routing decision is made by the route() function in graph.py:

- If state['escalate'] is True → route to escalate node → human agent handles the query

- If state['escalate'] is False → route to answer node → LLM handles the query

The escalate flag is set in process_node based on this condition:

```
if not context or len(context.strip()) < 20:  
    state['escalate'] = True  
else:  
    state['escalate'] = False
```

6. HITL Implementation

6.1 Role of Human Intervention

Human-in-the-Loop (HITL) is a design pattern where a human agent is brought into an automated workflow at specific decision points. In this system, the human agent acts as a fallback when the bot fails to find a confident answer. This is critical for customer support applications where wrong answers can damage customer trust.

The human agent receives the original customer query and types a response directly into the terminal. This response is then delivered to the customer exactly as if it came from the bot, maintaining a seamless user experience.

6.2 Benefits of HITL

- Ensures customers always receive an answer, even for edge-case queries
- Human agents can handle complex, nuanced, or emotionally sensitive queries
- Builds trust by combining AI speed with human judgment
- Provides a safety net that prevents the bot from giving wrong or harmful answers
- Human responses can be collected to expand the knowledge base over time
-

6.3 Limitations of HITL

- Requires a human agent to be available and monitoring the terminal
- Introduces latency when a human must type a response manually
- Does not scale well if many queries require escalation simultaneously
- In this CLI implementation, only one conversation can be handled at a time
- A production system would route escalations to a ticketing system or live chat platform

7. Challenges and Trade-offs

7.1 Retrieval Accuracy vs Speed

FAISS provides approximate nearest neighbor search, which is extremely fast but may occasionally miss the most relevant chunk in favor of a slightly less relevant one. Exact search (using brute force) would be more accurate but much slower for large datasets. For this project, approximate search was chosen because the knowledge base is small and speed is prioritized.

7.2 Chunk Size vs Context Quality

Smaller chunks produce more precise retrieval but may miss important context that spans multiple sentences. Larger chunks capture more context but may include irrelevant content that confuses the LLM. The chosen chunk size of 500 characters with 50-character overlap was a balanced decision after considering the average length of a FAQ answer in the knowledge base.

7.3 Cost vs Performance

Using the Groq free tier with LLaMA 3.3 70B provides high-quality answers at zero cost, but is subject to rate limits (approximately 30 requests per minute). A paid OpenAI GPT-4 API would offer more reliability and higher rate limits but at significant cost. For an internship demo project, the free Groq tier is the optimal choice.

8. Testing Strategy

8.1 Testing Approach

The system was tested manually using a set of sample queries covering different categories. Each query was entered into the CLI and the response was evaluated for accuracy, relevance, and completeness. The HITL escalation was also tested by entering queries that do not exist in the knowledge base.

8.2 Sample Queries and Results

Query	Expected Behavior	Result
How do I reset my password?	Bot answers from FAQ	Answered correctly
What is the refund policy?	Bot answers from FAQ	Answered correctly

How do I cancel my subscription?	Bot answers from FAQ	Answered correctly
Why is my device showing offline?	Bot answers from FAQ	Answered correctly
I want to file a legal complaint	Escalated to human	Escalated correctly
Tell me a joke	Escalated to human	Escalated correctly

9. Future Enhancements

9.1 Multi-Document Support

The current system supports a single PDF knowledge base. Future versions could support multiple PDFs, Word documents, and web pages. LangChain's document loaders support a wide range of file formats including CSV, HTML, Notion, and Confluence pages. A document management interface could allow administrators to add and remove documents from the knowledge base without restarting the system.

9.2 Feedback Loop

A feedback mechanism could allow users to rate the quality of each answer with a thumbs up or thumbs down. Negative feedback could trigger automatic escalation and flag the query for knowledge base improvement. Over time, this feedback loop would continuously improve the quality of the knowledge base and retrieval accuracy.

9.3 Memory Integration

The current system has no memory of previous messages in the same conversation. Adding conversation memory using LangChain's ConversationBufferMemory would allow the bot to handle follow-up questions naturally. For example, after asking about password reset, the user could ask 'How long does it take?' and the bot would understand they are still referring to the password reset process.

9.4 Deployment

The current CLI interface can be upgraded to a web application using FastAPI for the backend and React or Streamlit for the frontend. The system can be containerized using Docker and deployed on cloud platforms such as AWS, Google Cloud, or Azure. A production deployment would also include authentication, logging, monitoring, and rate limiting to ensure reliability and security at scale.

--- End of Technical Documentation ---